RELATO DE EXPERIÊNCIA

DESENVOLVIMENTO DE UM SISTEMA WEB PARA O AGENDAMENTO DE ATENDIMENTOS ACADÊMICOS

CATEGORIA: Relato de experiência

ÁREA TEMÁTICA: Tecnologia e inovação aplicadas à gestão acadêmica

Matheus Endlich Vieira[1]; Pedro Henrique Pereira Lima[1]; Fernanda Vasconcelos do Carmo[1]

Carlos Eduardo Kobayashi Colombo[1]; Vinícius Ferreira Correa de Morais[1]; Bruno Matheus Rossow de Assis[1]

Ciência da Computação, Afya Centro Universitário Ji-Paraná

Professor orientador: José Rodolfo de Olivas

Resumo: O agendamento manual de atendimentos acadêmicos pode provocar conflitos de horário, dispersão de informações e dificuldade para acompanhar os resultados das demandas apresentadas pelos estudantes. Este trabalho relata o desenvolvimento de um sistema web para centralizar o fluxo de atendimento da coordenação de um curso de Ciência da Computação, desde a publicação das disponibilidades até o registro do resultado e a consolidação de indicadores. Adotou-se um processo incremental orientado por documentação, com requisitos rastreáveis, decisões arquiteturais registradas, testes automatizados e revisão dos fluxos no navegador. A solução foi implementada como um monólito modular em um monorepositório, com frontend em Vue 3 e TypeScript, API REST em NestJS e persistência em MariaDB por meio do Sequelize. Entre as regras críticas tratadas estão a prevenção de dupla reserva, a revalidação da disponibilidade dentro de transação, o isolamento dos dados de cada aluno, a preservação do histórico e a auditoria das operações administrativas. O produto resultante contempla os fluxos de aluno, professor e administrador, incluindo agenda, cancelamento, registro de atendimento, acompanhamento de demandas e quatro indicadores operacionais. Os resultados obtidos demonstram a implementação do fluxo vertical principal e a rastreabilidade entre requisitos, código, testes e evidências visuais. A integração real com o WhatsApp, a validação concorrente em um banco real e definições institucionais ainda pendentes foram mantidas como trabalhos futuros.

Palavras-chave: agendamento acadêmico; sistema web; arquitetura modular; concorrência; rastreabilidade.

Abstract: Manual scheduling of academic support meetings can lead to time conflicts, scattered information, and difficulty in tracking the outcomes of student requests. This paper reports the development of a web system that centralizes the support workflow of a Computer Science program, from publishing available time slots to recording outcomes and consolidating operational indicators. An incremental, documentation-driven process was adopted, with traceable requirements, recorded architectural decisions, automated tests, and browser-based workflow reviews. The solution was implemented as a modular monolith in a monorepository, using Vue 3 and TypeScript for the frontend, a NestJS REST API, and MariaDB with Sequelize for persistence. Critical rules include preventing double booking, revalidating availability within a transaction, isolating each student's data, preserving history, and auditing administrative operations. The resulting product supports student, professor, and administrator workflows, including schedules, cancellations, attendance records, request tracking, and four operational indicators. The results show the implementation of the main vertical workflow and traceability among requirements, code, tests, and visual evidence. Real WhatsApp integration, concurrency validation against a real database, and pending institutional definitions remain as future work.

Keywords: academic scheduling; web system; modular architecture; concurrency; traceability.

1 INTRODUÇÃO

O atendimento entre estudantes e a coordenação de um curso envolve atividades que ultrapassam a simples escolha de uma data. É necessário disponibilizar horários, impedir conflitos, identificar o responsável, registrar o assunto, acompanhar o comparecimento, documentar encaminhamentos e consolidar os resultados. Quando esse processo ocorre por mensagens ou registros dispersos, a confirmação depende de conferência manual, o histórico torna-se difícil de consultar e a gestão perde visibilidade sobre as demandas recorrentes.

A literatura indica que o contato com a orientação acadêmica possui relação relevante com a permanência e o progresso do estudante. Goemans e Kapinos (2024), ao analisarem 34.340 estudantes de uma rede de *community colleges*, identificaram associação entre maior frequência de encontros com orientadores e indicadores como persistência, créditos concluídos e antecipação da matrícula no semestre seguinte. O estudo também descreve o uso de uma plataforma de gestão para registrar data, horário, modalidade, assuntos discutidos e ações de continuidade, elementos próximos aos registros necessários no contexto deste projeto.

A tecnologia, entretanto, não resolve sozinha as dificuldades do atendimento. Kalamkarian *et al.* (2020), ao examinarem iniciativas de reformulação da orientação acadêmica em três instituições, observaram que ferramentas digitais podem tornar o agendamento e a documentação das sessões mais eficientes, mas precisam estar alinhadas às práticas e à estrutura organizacional. Angelia e Tambotoh (2024) também destacam, em um sistema de orientação acadêmica projetado com abordagem centrada no ser humano, a importância de reduzir a fragmentação da comunicação, preservar o histórico e ajustar a solução às expectativas dos usuários.

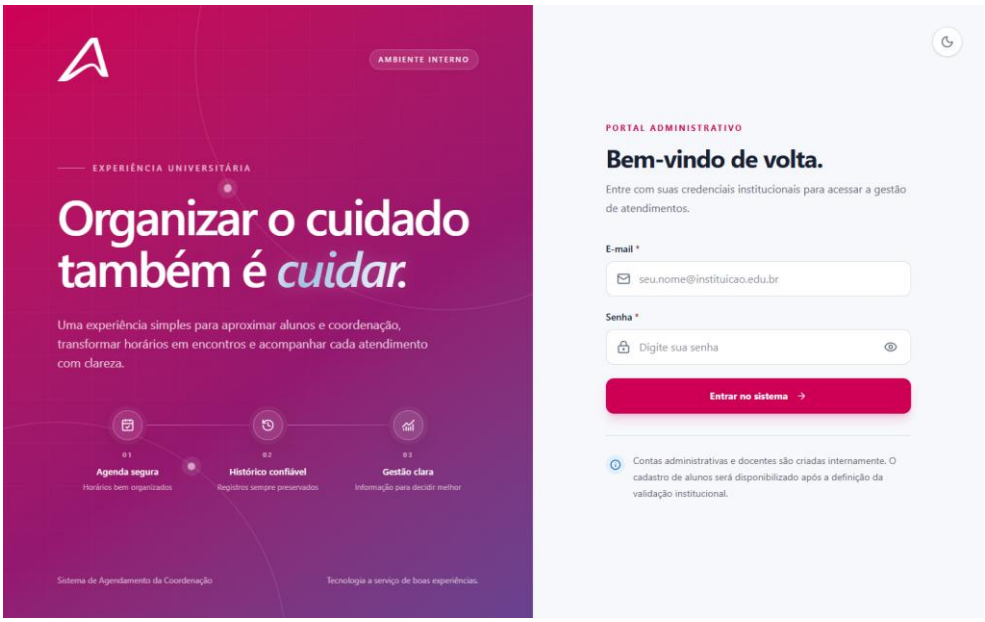
Outros trabalhos concentram-se na automação da orientação ou da escolha de disciplinas. Roushan *et al.* (2014) propõem uma ferramenta web de apoio à orientação e matrícula capaz de reduzir atividades repetitivas e o uso de papel. Embora compartilhe a intenção de organizar um processo acadêmico, essa solução está voltada à recomendação e ao registro de disciplinas. O sistema descrito neste artigo diferencia-se por tratar o encontro entre estudante e coordenação como um fluxo completo: disponibilidade, reserva concorrente, cancelamento, comparecimento, resultado, continuidade, auditoria e indicadores.

O problema abordado surgiu da necessidade de centralizar os atendimentos da coordenação de um curso de Ciência da Computação. A solução deveria atender a três perfis principais. O aluno precisaria consultar horários livres, confirmar uma reserva e acompanhar os próprios compromissos. O professor deveria publicar disponibilidades e registrar a realização ou a ausência. A administração precisaria de visão global, instrumentos de auditoria e indicadores derivados dos atendimentos efetivamente registrados.

O objetivo geral foi desenvolver um sistema web capaz de organizar o fluxo completo do atendimento acadêmico, preservando segurança, histórico e consistência dos dados. Como objetivos específicos, buscou-se implementar autenticação e autorização por perfil e permissão; permitir a publicação e o bloqueio de disponibilidades; confirmar agendamentos de forma concorrente e transacional; oferecer cancelamento auditável; registrar atendimento, ausência, resultado e continuidade da demanda; apresentar indicadores operacionais; e manter requisitos, decisões, implementação e testes rastreáveis.

O escopo concentrou-se no fluxo vertical mínimo do produto. Integrações acadêmicas externas, relatórios avançados, múltiplos canais de notificação e hospedagem definitiva não fizeram parte desta etapa. Essa delimitação permitiu priorizar regras que afetam diretamente a integridade do agendamento e a privacidade dos usuários.

Figura 1 - Tela de acesso ao sistema



Fonte: elaborado pelos autores (2026).

2 MATERIAIS E MÉTODOS OU PROCEDIMENTOS METODOLÓGICOS

2.1 Caracterização do trabalho

O trabalho caracteriza-se como relato de experiência sobre o desenvolvimento tecnológico de uma solução para um problema operacional. A avaliação combinou testes automatizados, inspeção dos fluxos no navegador, análise do histórico do repositório e apresentação de uma versão estável ao professor orientador.

O desenvolvimento adotou uma abordagem incremental orientada por documentação. No desenvolvimento incremental, o sistema é construído em partes sucessivas, cada uma entregando funcionalidade utilizável e permitindo incorporar o retorno obtido nas etapas anteriores (SOMMERVILLE, 2011; PRESSMAN; MAXIM, 2016). Antes do primeiro *scaffold* executável, o repositório já possuía documentos de escopo, requisitos, arquitetura, modelo de domínio, contrato inicial da API e questões ainda pendentes. Os requisitos foram numerados e relacionados à implementação e aos testes, permitindo rastrear os principais requisitos do MVP até sua origem.

As decisões técnicas relevantes foram registradas como *Architecture Decision Records* (ADRs), documentos que preservam o contexto, a escolha e suas consequências (NYGARD, 2011). Uma decisão superada não foi apagada; um novo registro passou a substituí-la, mantendo a justificativa histórica. Esse procedimento foi aplicado, por exemplo, quando a proposta inicial de Flask e SQLAlchemy foi substituída pela stack baseada em Node.js, NestJS, Sequelize e Umzug.

Após a criação dos projetos frontend e backend, as funcionalidades foram organizadas em etapas: identidade e autorização; disponibilidade e agendamento; cancelamento; registro de atendimento; evolução da demanda; dashboard; e preparação para demonstração. Com a formalização do fluxo Git, cada alteração passou a ser desenvolvida em uma branch curta, integrada por Pull Request em develop e promovida para main apenas quando integrava uma versão estável. O ambiente de demonstração da main foi isolado em um *worktree* próprio e em um banco distinto do desenvolvimento.

2.2 Materiais e tecnologias

A solução foi construída em um monorepositório executado com Node.js 24 e pnpm. O frontend utiliza Vue 3, TypeScript, Vite, Pinia, Vue Router e Tailwind CSS. O backend utiliza NestJS e TypeScript e expõe uma API REST com representações em JSON. A persistência emprega MariaDB, Sequelize 6, sequelize-typescript e migrations controladas pelo Umzug.

A organização em camadas separa apresentação, regras de negócio e acesso a dados, reduzindo o acoplamento entre responsabilidades (FOWLER, 2002). A comunicação entre cliente e servidor segue princípios do estilo REST (FIELDING, 2000). O monólito modular mantém uma única unidade de implantação, mas organiza o código em módulos com fronteiras explícitas, evitando a complexidade operacional de serviços distribuídos enquanto o domínio ainda se consolida (FOWLER, 2015).

As entidades centrais do modelo são usuário, disponibilidade, agendamento e atendimento. O modelo também contempla sessões, permissões, navegação, parâmetros, notificações e auditoria. Alterações estruturais do banco são versionadas exclusivamente por migrations; a sincronização automática do ORM permanece desabilitada.

2.3 Regras críticas de consistência e segurança

A confirmação do agendamento foi tratada como operação transacional. Uma transação agrupa operações que devem ser aplicadas integralmente ou não ser aplicadas, preservando a consistência diante de falhas e execuções simultâneas (GRAY; REUTER, 1993). Dentro da transação, o serviço bloqueia as linhas relevantes com a estratégia equivalente a SELECT ... FOR UPDATE (MARIADB, [s.d.]), revalida a disponibilidade e procura sobreposições antes de inserir a reserva. A tentativa de notificação acontece apenas depois do *commit*, de forma que uma falha no canal de mensagens não desfaça um agendamento válido.

A autenticação mantém sessões revogáveis em cookie HttpOnly, e a autorização é recalculada no backend em cada operação protegida. As senhas são armazenadas com Argon2id, função projetada para elevar o custo de ataques de força bruta por meio do uso de memória (BIRYUKOV *et al.*, 2021). As práticas de controle de acesso, sessão e proteção de dados foram orientadas pelo OWASP ASVS (OWASP, 2025). O tratamento de dados pessoais considerou os princípios da Lei Geral de Proteção de Dados Pessoais (BRASIL, 2018), e a interface foi desenvolvida com atenção a navegação por teclado, foco visível, rótulos e mensagens textuais, conforme as WCAG 2.2 (W3C, 2024).

2.4 Procedimentos de verificação

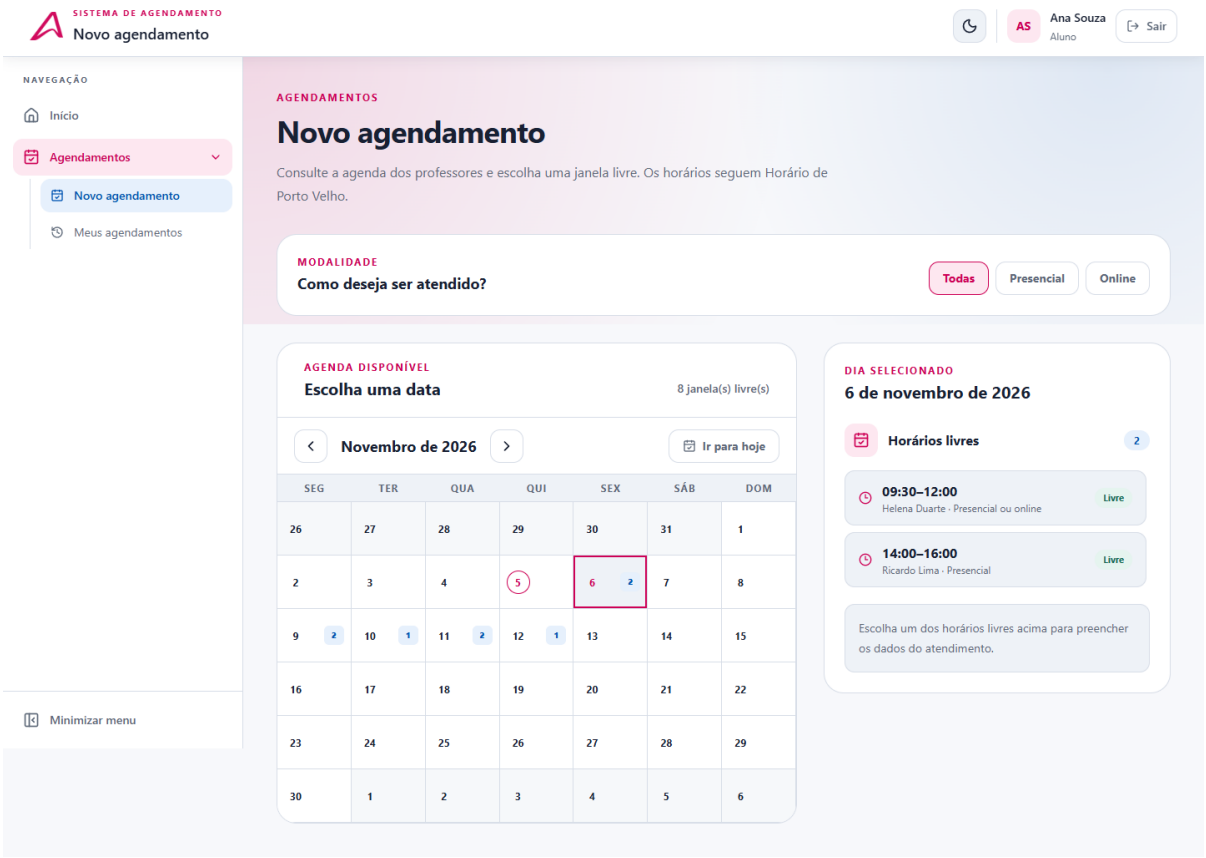
A qualidade foi verificada em quatro níveis complementares:

1. lint e formatação automática para padronização do código;
2. checagem estática de tipos no frontend;
3. testes automatizados de serviços, migrations, autorização e componentes;

4. revisão manual dos fluxos dos três perfis no navegador.

O conjunto mínimo de testes contemplou concorrência de confirmações, autorização entre alunos, conflitos de horário, independência entre notificação e transação, indicadores, auditoria, acessibilidade dos componentes principais e reversibilidade das migrations. Os dados das demonstrações são fictícios e recriáveis. As contas usam o domínio reservado @demo.test, CPFs deliberadamente inválidos e telefones reservados, evitando a exposição de dados pessoais nas evidências visuais.

Figura 2 - Consulta de horários no ambiente de demonstração



Fonte: elaborado pelos autores (2026).

3 RESULTADOS E DISCUSSÃO

3.1 Arquitetura e organização do domínio

O sistema resultante foi estruturado como monólito modular em um único repositório. O frontend concentra interface, navegação e estado de apresentação. O backend concentra regras de negócio, autorização e transações. Controllers tratam a fronteira HTTP e delegam a serviços de aplicação, enquanto todo acesso ao banco passa pelo Sequelize. A integração de notificações permanece atrás de uma porta própria, impedindo que módulos de domínio dependam diretamente da SDK de um provedor de WhatsApp.

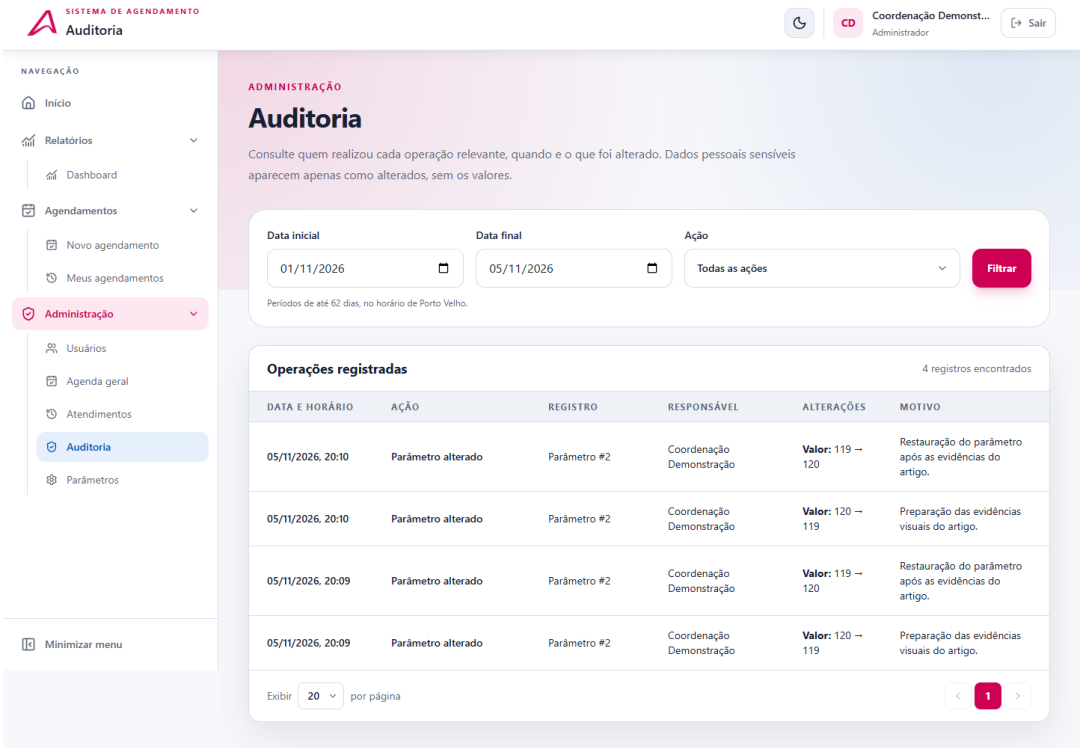
Na demonstração remota, o backend serve também o build da aplicação de página única. O serviço fica restrito ao endereço de loopback e é exposto por HTTPS por meio de um Cloudflare Tunnel. A porta do MariaDB não é publicada. Essa configuração atende à apresentação temporária, mas não substitui uma infraestrutura definitiva de produção.

Um professor publica janelas de disponibilidade e define as modalidades permitidas. Um aluno confirma um agendamento dentro de uma dessas janelas. Quando o compromisso é realizado, o agendamento origina um registro de atendimento, que recebe o resultado da demanda e, quando necessário, um histórico de continuidade. Os indicadores não são persistidos como contadores independentes: são calculados a partir dos registros operacionais, reduzindo o risco de divergência entre histórico e dashboard.

3.2 Autenticação, autorização e auditoria

O aluno consulta apenas os próprios agendamentos e não recebe observações internas da coordenação. Professores acessam os registros associados às suas disponibilidades. O administrador possui visão global, porém operações sensíveis continuam exigindo permissões explícitas e autoria auditável. Proteções de rota no frontend melhoram a experiência de navegação, mas não substituem a autorização do servidor.

Figura 3 - Trilha de auditoria administrativa



Fonte: elaborado pelos autores (2026).

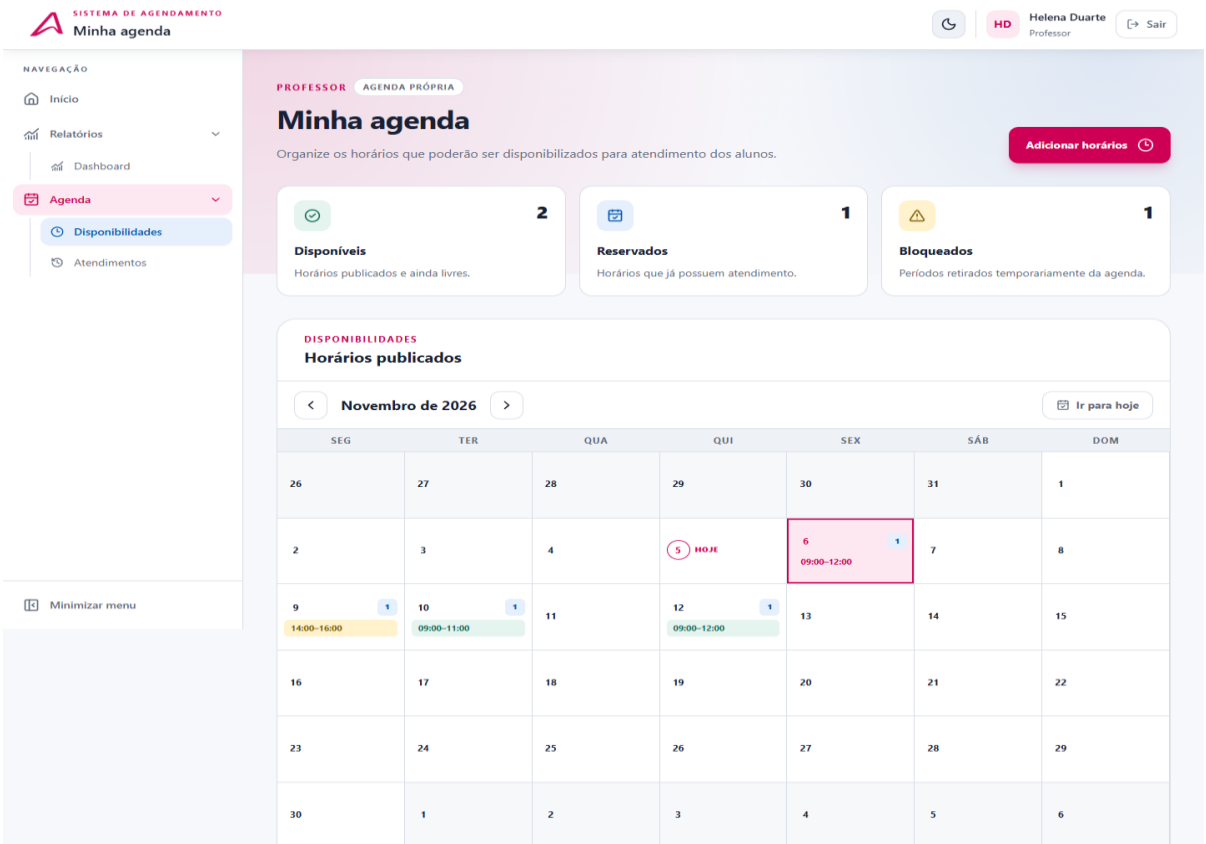
Cancelamentos administrativos e alterações de parâmetros registram ator, data e motivo. A trilha de auditoria é separada dos logs técnicos, pois atende a uma finalidade distinta: explicar quem realizou uma operação administrativa relevante e em qual contexto. Essa separação preserva evidências de negócio sem confundi-las com mensagens destinadas ao diagnóstico da aplicação.

3.3 Publicação de disponibilidades

O professor pode publicar uma janela para uma data específica ou preparar várias janelas para os dias selecionados de uma semana. Cada janela possui início, término e uma ou mais modalidades permitidas. O backend valida a ordem dos horários, o timezone institucional e a inexistência de sobreposição antes da persistência. Horários ainda não reservados podem ser bloqueados e desbloqueados pelo responsável.

A visualização mensal reúne as janelas em um calendário e apresenta os detalhes do dia selecionado em um painel lateral. O componente foi adaptado para telas menores, preservando navegação por teclado, foco visível e estados de carregamento, vazio e erro. A seleção de vários dias e várias faixas de horário reduz operações repetitivas sem retirar do backend a validação final.

Figura 4 - Agenda mensal do professor



Fonte: elaborado pelos autores (2026).

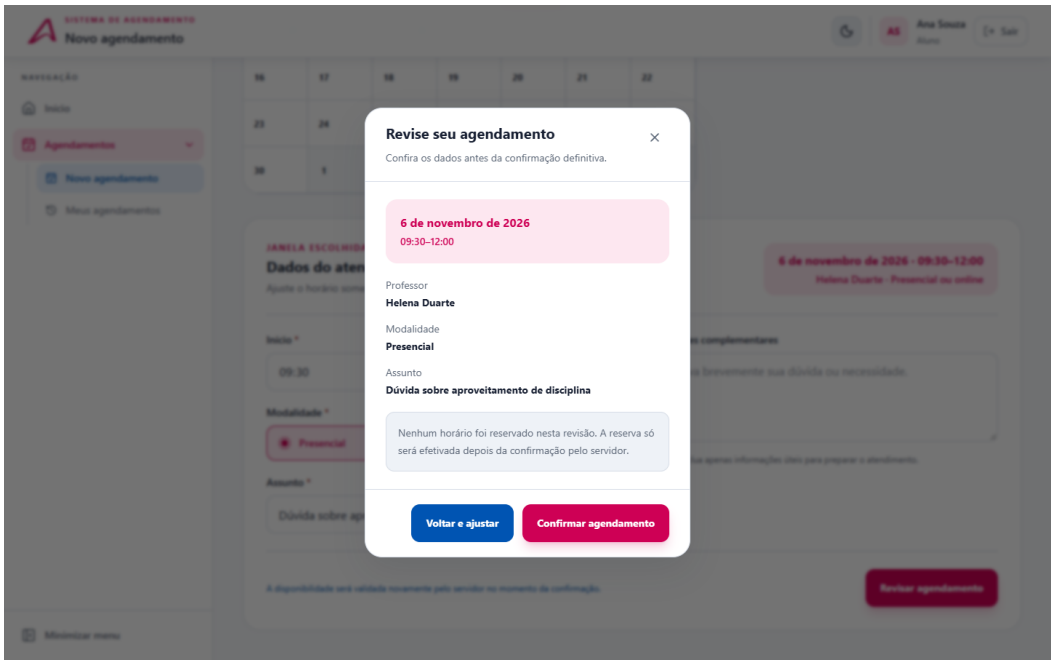
3.4 Confirmação concorrente do agendamento

A prevenção de dupla reserva constitui a principal regra de consistência do fluxo. Uma checagem apenas no frontend seria insuficiente, pois dois alunos poderiam selecionar o mesmo intervalo antes de qualquer resposta do servidor. Por isso, a confirmação ocorre em transação e revalida tanto a disponibilidade quanto os compromissos conflitantes do aluno.

Dentro da transação, a linha do aluno e a linha da disponibilidade são lidas com bloqueio pessimista. Quando o bloqueio do banco funciona conforme projetado, uma segunda confirmação concorrente deve aguardar o encerramento da primeira e, ao revalidar, encontrar o intervalo ocupado. Não existe uma restrição de unicidade simples para períodos sobrepostos; a consistência depende da serialização do acesso e da revalidação no serviço.

Quando a reserva é aceita, o servidor gera um protocolo único. O protocolo permite identificar o compromisso sem expor dados pessoais na URL. A tentativa de notificação é iniciada depois da conclusão da transação. Assim, indisponibilidade do provedor não altera o estado confirmado da reserva.

Figura 5 - Revisão do agendamento antes da confirmação



Fonte: elaborado pelos autores (2026).

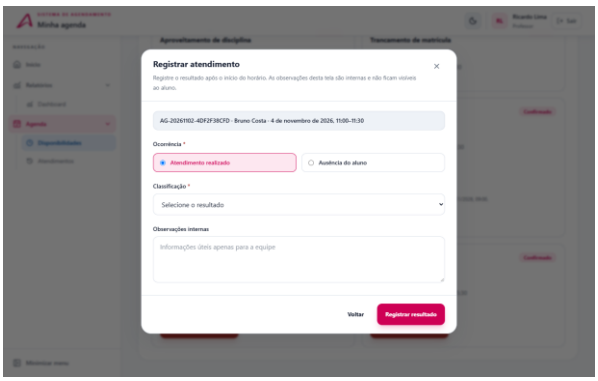
3.5 Cancelamento, atendimento e continuidade

Aluno e professor podem cancelar um agendamento confirmado quando a antecedência configurada permite, sempre informando um motivo. O administrador

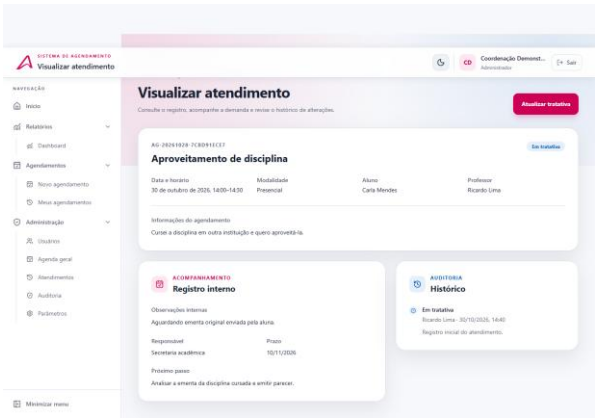
também pode cancelar, mas precisa confirmar a própria senha. O cancelamento é lógico e preserva o histórico; cancelamento e ausência não são classificados como atendimentos resolvidos.

Depois do horário marcado, o professor registra a realização ou a ausência. Um atendimento realizado pode ser classificado como resolvido, não resolvido ou em tratativa. Demandas em tratativa exigem informação mínima de continuidade e podem receber novas evoluções até a resolução ou o encerramento sem solução. Os estados finais não são reabertos no escopo do MVP.

Figura 6 - Registro e acompanhamento do atendimento



(a) Registro do resultado pelo professor.



(b) Acompanhamento da demanda pela administração.

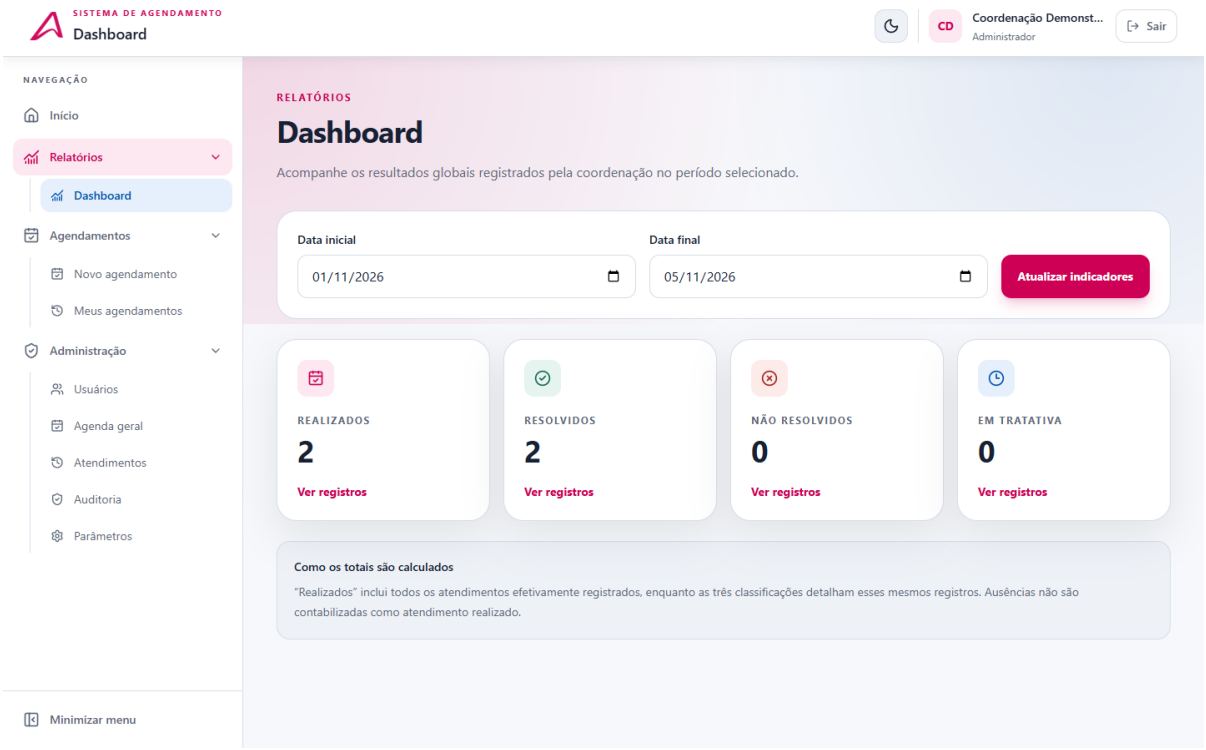
Fonte: elaborado pelos autores (2026).

3.6 Indicadores operacionais

O dashboard apresenta quatro totais: atendimentos realizados, resolvidos, não resolvidos e em tratativa. Os dados podem ser filtrados por período. Para o professor, o backend restringe o cálculo às próprias disponibilidades; para a administração, considera o conjunto autorizado. A interface permite abrir os registros que compõem cada número, favorecendo a conferência do resultado.

A taxa percentual de resolução não foi incluída porque seu denominador ainda depende de definição institucional. Manter apenas os totais aprovados evita apresentar uma métrica aparentemente precisa com significado ambíguo.

Figura 7 - Dashboard de atendimentos



Fonte: elaborado pelos autores (2026).

3.7 Verificação e rastreabilidade

O fluxo vertical proposto foi implementado desde a publicação da disponibilidade até a consolidação dos indicadores. O aluno consulta horários, confirma uma reserva, recebe um protocolo e acompanha seus compromissos. O professor mantém a agenda e registra o resultado. A administração possui visão global, parâmetros operacionais e trilha de auditoria.

O dossiê de desenvolvimento mantém uma matriz que relaciona requisitos do MVP a Pull Requests, decisões arquiteturais, arquivos principais, testes e figuras. No recorte documentado até 31 de outubro de 2026, o repositório apresentava a evolução resumida no Quadro 1.

Quadro 1 - Indicadores do desenvolvimento até 31 de outubro de 2026

Indicador	Valor
Início da documentação	29 de agosto de 2026
Commits de trabalho	29
Pull Requests	27, sendo 25 integrados e 2 em revisão no recorte
Versões estáveis publicadas	5

Indicador	Valor
Decisões arquiteturais	32, sendo 2 superadas
Módulos de domínio	10
Tabelas e migrations	20 tabelas e 28 migrations
Endpoints REST e telas	43 endpoints e 15 telas
Testes automatizados	evolução de 86/66 para 188/126 no backend/frontend

Fonte: elaborado pelos autores a partir do histórico do repositório (2026).

Uma versão estável promovida em 28 de outubro de 2026, com 176 testes no backend e 116 no frontend, foi apresentada ao professor orientador em 31 de outubro. A navegação pelos três perfis revelou problemas que não eram evidentes nos testes unitários, como checagem prematura de permissões, associação de menu a uma rota ainda inexistente e perda do destaque visual do horário selecionado. As correções reforçam que automação e revisão de uso são complementares.

O teste de concorrência exige uma ressalva. Ele executa duas confirmações sobre o mesmo intervalo e verifica que somente uma é aceita, mas substitui o MariaDB por uma trava simulada que imita o bloqueio de linha. Portanto, comprova a lógica do serviço - bloquear, revalidar e somente então inserir - , mas não o comportamento do banco sob concorrência real. Um teste de integração com duas conexões reais ao MariaDB permanece necessário.

Os resultados dialogam com os trabalhos relacionados. Assim como no relato de Kalamkarian *et al.* (2020), o sistema unifica agendamento e documentação do atendimento, mas acrescenta regras transacionais, autorização detalhada e auditoria administrativa. Em relação a Angelia e Tambotoh (2024), compartilha a preocupação com histórico e experiência dos usuários, embora a avaliação do Magrin SAC ainda esteja concentrada em testes técnicos e revisão funcional. O projeto também vai além da automação de orientação descrita por Roushan *et al.* (2014), pois acompanha o resultado e a continuidade da demanda após a reserva.

3.8 Limitações e trabalhos futuros

A integração real com o WhatsApp permanece adiada. O módulo de notificações e a regra de execução posterior à transação estão preparados, mas ainda são necessários conta do provedor, consentimento, templates, política de reenvio e tratamento operacional de falhas.

Também permanecem pendentes a fórmula institucional do percentual de resolução, o limite de agendamentos futuros por aluno, as janelas mínima e máxima de antecedência, a alteração administrativa de data e horário, a recuperação de senha e o autocadastro com validação institucional. Categorias oficiais de atendimento, histórico acadêmico, local ou link da reunião, política de retenção e hospedagem definitiva exigem decisões adicionais.

Sob a perspectiva de avaliação, faltam o teste concorrente com MariaDB real e uma pesquisa estruturada com usuários finais. A associação entre encontros de orientação e resultados acadêmicos encontrada por Goemans e Kapinos (2024) não pode ser atribuída ao presente sistema sem coleta longitudinal. Os indicadores do MVP descrevem o atendimento realizado; eles não demonstram, por si sós, impacto sobre retenção, evasão ou desempenho.

O sistema ainda precisa de observabilidade, política institucional de retenção, rotina de backup e restauração integralmente testada e infraestrutura definitiva de produção. A publicação por túnel atende demonstrações temporárias, mas não constitui ambiente de produção.

4 CONSIDERAÇÕES FINAIS OU CONCLUSÃO

Este trabalho apresentou o desenvolvimento de um sistema web para organizar os atendimentos acadêmicos da coordenação de um curso de Ciência da Computação. A solução centraliza disponibilidade, agendamento, cancelamento, registro do atendimento, evolução da demanda, auditoria e indicadores em um fluxo único.

A arquitetura de monólito modular ofereceu separação de responsabilidades sem introduzir a complexidade operacional de uma solução distribuída. A confirmação transacional com bloqueio de linha e revalidação no backend tratou o risco de dupla reserva na lógica da aplicação. A autorização por escopo, a preservação do histórico e a auditoria formaram uma base compatível com a sensibilidade dos dados acadêmicos.

O processo orientado por documentação permitiu relacionar requisitos, decisões, código, testes e evidências visuais. Como resultado, o MVP cobre o fluxo principal e fornece uma base evolutiva para integrações e regras institucionais ainda pendentes. Ao mesmo tempo, o trabalho reconhece que a validação técnica não substitui avaliação com usuários e que o comportamento concorrente ainda precisa ser comprovado em integração com o MariaDB real.

As próximas etapas devem priorizar a validação institucional das regras abertas, o teste concorrente no banco, a avaliação de usabilidade com alunos e professores, a operação segura e a integração de notificações sem comprometer a transação principal.

REFERÊNCIAS

ANGELIA, A.; TAMBOTOH, J. J. C. Designing an Academic Advising Information System: A Human-Centered Design Approach. **JUSIFO: Jurnal Sistem Informasi**, v. 10, n. 2, p. 135-142, 2024. DOI: https://doi.org/10.19109/jusifo.v10i2.25258.

BIRYUKOV, A.; DINU, D.; KHOVRATOVICH, D.; JOSEFSSON, S. **Argon2 memory-hard function for password hashing and proof-of-work applications**. RFC 9106. Internet Engineering Task Force, 2021. Disponível em: https://www.rfc-editor.org/info/rfc9106. Acesso em: 5 nov. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais. Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 12 nov. 2026.

FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Ciência da Computação) - University of California, Irvine, 2000. Disponível em: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm. Acesso em: 12 nov. 2026.

FOWLER, M. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

FOWLER, M. **MonolithFirst**. 2015. Disponível em: https://martinfowler.com/bliki/MonolithFirst.html. Acesso em: 5 nov. 2026.

GOEMANS, M.; KAPINOS, B. A quantitative study of community college student-advisor appointments and student success metrics. **NACADA Journal**, v. 44, n. 1, p. 38-54, 2024. DOI: https://doi.org/10.12930/NACADA-23-31.

GRAY, J.; REUTER, A. **Transaction processing: concepts and techniques**. San Francisco: Morgan Kaufmann, 1993.

KALAMKARIAN, H. S.; PELLEGRINO, L.; LOPEZ, A.; BARNETT, E. A. **Lessons learned from advising redesigns at three colleges**. New York: Community College Research Center, Teachers College, Columbia University, 2020. Disponível em: https://files.eric.ed.gov/fulltext/ED609264.pdf. Acesso em: 11 nov. 2026.

MAGRIN-SAC. **Sistema web de agendamento de atendimentos acadêmicos**. Repositório do projeto, 2026. Disponível em: https://github.com/Math3us11/magrin-sac. Acesso em: 12 nov. 2026.

MARIADB. **FOR UPDATE**. MariaDB Server Documentation, [s.d.]. Disponível em: https://mariadb.com/docs/server/reference/sql-statements/data-manipulation/selecting-data/for-update. Acesso em: 5 nov. 2026.

NYGARD, M. **Documenting architecture decisions**. 2011. Disponível em: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions. Acesso em: 5 nov. 2026.

OWASP. **Application Security Verification Standard 5.0.0**. OWASP Foundation, 2025. Disponível em: https://asvs.owasp.org/. Acesso em: 5 nov. 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

ROUSHAN, T.; CHAKI, D.; HASDAK, O.; CHOWDHURY, M. S.; RASEL, A. A.; RAHMAN, M. A.; ARIF, H. University course advising: overcoming the challenges using decision support system. In: **INTERNATIONAL CONFERENCE ON COMPUTER AND INFORMATION TECHNOLOGY**, 16., 2014, Khulna. Anais [...]. Piscataway: IEEE, 2014. p. 13-18. DOI: https://doi.org/10.1109/ICCITechn.2014.6997355.

SOMMERVILLE, I. **Engenharia de software**. 9. ed. São Paulo: Pearson, 2011.

W3C. **Web Content Accessibility Guidelines 2.2**. W3C Recommendation, 2024. Disponível em: https://www.w3.org/TR/WCAG22/. Acesso em: 5 nov. 2026.